UNIVERSIDAD NACIONAL DE COLOMBIA

SEDE MEDELLÍN

PROGRAMACIÓN ORIENTADA A OBJETOS

Docente: Walter Hugo Arboleda Mazo

Actividad 3 — *Valor 10% Individual*

Texto guía: *Ejercicios de Programación Orientada a Objetos con Java y UML* - Leonardo Bermón Angarita.

INTRODUCCIÓN

El presente informe documenta el desarrollo de la **Actividad 3** de la asignatura Programación Orientada a Objetos (POO). El objetivo principal es la implementación de interfaces gráficas de usuario (GUI) e interactividad mediante la gestión de eventos utilizando la biblioteca estándar **Tkinter** de **Python**, aplicando los conceptos fundamentales de la POO explicados en los ejercicios 8.2 y 8.3 del libro guía.

1. PUNTO 1: EJERCICIO 8.2 (Componentes GUI y Parte Propuesta)

1.1 Descripción del Problema

Se requiere desarrollar una aplicación gráfica que permita calcular el

Índice de Masa Corporal (IMC) de un paciente e interprete su condición de salud. Adicionalmente, como **parte propuesta**, se integra la funcionalidad para calcular el **Porcentaje de Grasa Corporal** utilizando la fórmula de Deurenberg y se implementa una **Barra de Menú superior** (con opciones de limpieza, ayuda y salida).

1.2 Código Fuente en Python (`ejercicio_8_2.py`)

```
"""
Universidad Nacional de Colombia - Programación Orientada
Actividad 1 (Ejercicio 8.2 - Componentes GUI y Menús)
Docente: Walter Hugo Arboleda Mazo
"""

import tkinter as tk
from tkinter import ttk, messagebox

class VentanaIMC(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Actividad 1 - Ejercicio 8.2: Cálculo
        self.geometry("480x520")
        self.resizable(False, False)

        # Barra de Menú (Parte Propuesta)
        self.crear_menu()

        # Contenedor principal
        main_frame = ttk.Frame(self, padding="20 20 20 20
        main_frame.pack(fill=tk.BOTH, expand=True)

        # Encabezado
        lbl_titulo = ttk.Label(main_frame, text="Calculad
        lbl_titulo.grid(row=0, column=0, columnspan=2, pa

        # Entradas de datos (Formulario)
        ttk.Label(main_frame, text="Nombre completo:").gr
        self.txt_nombre = ttk.Entry(main_frame, width=25)
```

```
        self.txt_nombre.grid(row=1, column=1, sticky=tk.E

        ttk.Label(main_frame, text="Edad (años):").grid(r
        self.txt_edad = ttk.Entry(main_frame, width=25)
        self.txt_edad.grid(row=2, column=1, sticky=tk.E,

        ttk.Label(main_frame, text="Masa / Peso (kg):").g
        self.txt_peso = ttk.Entry(main_frame, width=25)
        self.txt_peso.grid(row=3, column=1, sticky=tk.E,

        ttk.Label(main_frame, text="Estatura (m):").grid(
        self.txt_estatura = ttk.Entry(main_frame, width=2
        self.txt_estatura.grid(row=4, column=1, sticky=tk

        ttk.Label(main_frame, text="Género:").grid(row=5,
        self.cmb_genero = ttk.Combobox(main_frame, values
        self.cmb_genero.set("Hombre")
        self.cmb_genero.grid(row=5, column=1, sticky=tk.E

        # Botones de acción
        btn_frame = ttk.Frame(main_frame)
        btn_frame.grid(row=6, column=0, columnspan=2, pad

        btn_calcular = ttk.Button(btn_frame, text="Calcul
        btn_calcular.pack(side=tk.LEFT, padx=5)

        btn_limpiar = ttk.Button(btn_frame, text="Limpiar
        btn_limpiar.pack(side=tk.LEFT, padx=5)

        # Área de Resultados
        lbl_res_header = ttk.Label(main_frame, text="Resu
        lbl_res_header.grid(row=7, column=0, columnspan=2

        self.txt_resultados = tk.Text(main_frame, height=
        self.txt_resultados.grid(row=8, column=0, columns

    def crear_menu(self):
        menubar = tk.Menu(self)

        menu_archivo = tk.Menu(menubar, tearoff=0)
```

```
        menu_archivo.add_command(label="Limpiar Formulari
        menu_archivo.add_separator()
        menu_archivo.add_command(label="Salir", command=s
        menubar.add_cascade(label="Archivo", menu=menu_ar

        menu_ayuda = tk.Menu(menubar, tearoff=0)
        menu_ayuda.add_command(label="Acerca de...", comm
        menubar.add_cascade(label="Ayuda", menu=menu_ayud

        self.config(menu=menubar)

    def calcular(self):
        try:
            nombre = self.txt_nombre.get().strip()
            edad = int(self.txt_edad.get())
            peso = float(self.txt_peso.get())
            estatura = float(self.txt_estatura.get())
            genero = self.cmb_genero.get()

            if estatura <= 0 or peso <= 0 or edad <= 0:
                raise ValueError("Valores numéricos deben

            # Cálculo de IMC
            imc = peso / (estatura ** 2)

            # Clasificación de acuerdo a la tabla del lib
            if imc < 16:
                estado_imc = "Delgadez severa"
            elif imc < 17:
                estado_imc = "Delgadez moderada"
            elif imc < 18.5:
                estado_imc = "Delgadez leve"
            elif imc < 25:
                estado_imc = "Peso normal"
            elif imc < 30:
                estado_imc = "Sobrepeso"
            elif imc < 35:
                estado_imc = "Obesidad leve"
            elif imc < 40:
                estado_imc = "Obesidad moderada"
```

```
            else:
                estado_imc = "Obesidad mórbida"

            # Porcentaje de Grasa Corporal (Fórmula Deure
            val_sexo = 1 if genero == "Hombre" else 0
            porcentaje_grasa = (1.20 * imc) + (0.23 * eda

            # Formatear salida
            res_text = f"Paciente: {nombre if nombre else
            res_text += f"--------------------------------
            res_text += f"IMC Calculado: {imc:.2f} kg/m²\
            res_text += f"Diagnóstico IMC: {estado_imc}\n
            res_text += f"% Grasa Corporal Estimado: {por

            self.txt_resultados.delete("1.0", tk.END)
            self.txt_resultados.insert(tk.END, res_text)

        except ValueError as e:
            messagebox.showerror("Error de Entrada", f"Po

    def limpiar(self):
        self.txt_nombre.delete(0, tk.END)
        self.txt_edad.delete(0, tk.END)
        self.txt_peso.delete(0, tk.END)
        self.txt_estatura.delete(0, tk.END)
        self.cmb_genero.set("Hombre")
        self.txt_resultados.delete("1.0", tk.END)

    def mostrar_acerca_de(self):
        messagebox.showinfo("Acerca de", "Universidad Nac

if __name__ == "__main__":
    app = VentanaIMC()
    app.mainloop()
```

1.3 Diagramas UML (Ejercicio 8.2)

Diagrama de Clases UML

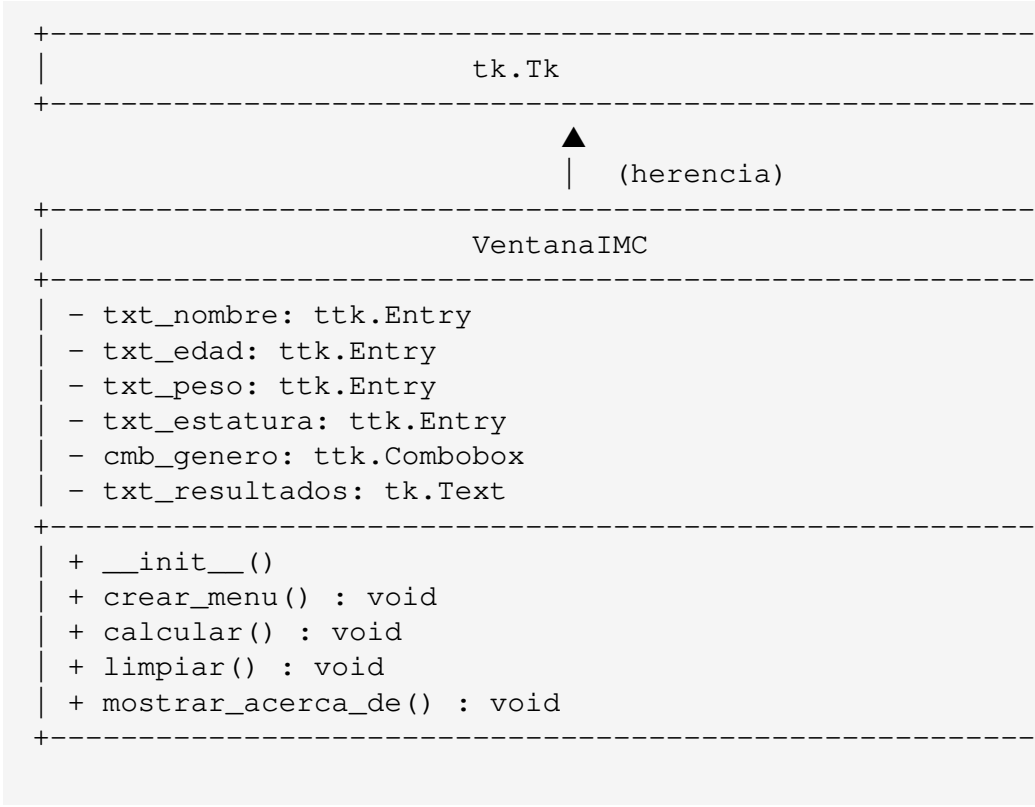
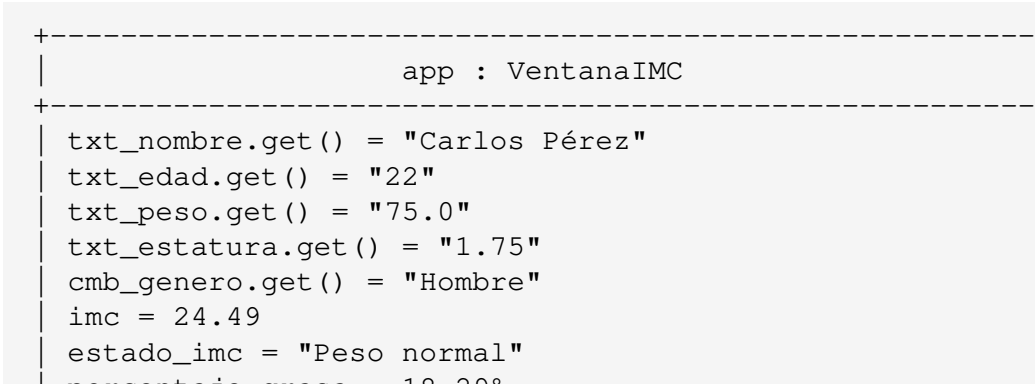


Diagrama de Objetos UML



```
+-------------------------------------------------------------
```

2. PUNTO 2: EJERCICIO 8.3 (Gestión de Eventos y Validaciones)

2.1 Descripción del Problema

Se implementa el procesamiento dinámico de eventos de teclado mediante `bind()`. La aplicación valida la entrada de datos numéricos en tiempo real mientras el usuario escribe (`<KeyRelease>`) y provee accesos directos de teclado (`<Return>` para calcular y `<Escape>` para limpiar).

2.2 Código Fuente en Python (`ejercicio_8_3.py`)

```
"""
Universidad Nacional de Colombia - Programación Orientada
Actividad 1 (Ejercicio 8.3 - Gestión de Eventos y Validac
Docente: Walter Hugo Arboleda Mazo
"""

import tkinter as tk
from tkinter import ttk, messagebox

class VentanaEventos(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Actividad 1 - Ejercicio 8.3: Gestión
        self.geometry("500x480")

        main_frame = ttk.Frame(self, padding="20 20 20 20
        main_frame.pack(fill=tk.BOTH, expand=True)
```

```
        lbl_titulo = ttk.Label(main_frame, text="Gestión
        lbl_titulo.grid(row=0, column=0, columnspan=2, pa

        lbl_info = ttk.Label(main_frame, text="Atajos: [E
        lbl_info.grid(row=1, column=0, columnspan=2, pady

        ttk.Label(main_frame, text="Masa / Peso (kg):").g
        self.txt_peso = ttk.Entry(main_frame, width=25)
        self.txt_peso.grid(row=2, column=1, sticky=tk.E,
        self.txt_peso.bind("<KeyRelease>", self.validar_e

        ttk.Label(main_frame, text="Estatura (m):").grid(
        self.txt_estatura = ttk.Entry(main_frame, width=2
        self.txt_estatura.grid(row=3, column=1, sticky=tk
        self.txt_estatura.bind("<KeyRelease>", self.valid

        self.lbl_status = ttk.Label(main_frame, text="Est
        self.lbl_status.grid(row=4, column=0, columnspan=

        btn_frame = ttk.Frame(main_frame)
        btn_frame.grid(row=5, column=0, columnspan=2, pad

        self.btn_calcular = ttk.Button(btn_frame, text="C
        self.btn_calcular.pack(side=tk.LEFT, padx=5)

        self.btn_limpiar = ttk.Button(btn_frame, text="Li
        self.btn_limpiar.pack(side=tk.LEFT, padx=5)

        self.txt_salida = tk.Text(main_frame, height=8, w
        self.txt_salida.grid(row=6, column=0, columnspan=

        self.bind("<Return>", lambda event: self.ejecutar
        self.bind("<Escape>", lambda event: self.limpiar_

    def validar_entrada_peso(self, event):
        val = self.txt_peso.get()
        if not val.replace('.', '', 1).isdigit() and val
            self.lbl_status.config(text="⚠  Error: El pes
        else:
            self.lbl_status.config(text="✓ Entrada de pes
```

```
    def validar_entrada_estatura(self, event):
        val = self.txt_estatura.get()
        if not val.replace('.', '', 1).isdigit() and val
            self.lbl_status.config(text="⚠  Error: La est
        else:
            self.lbl_status.config(text="✓ Entrada de est

    def ejecutar_calculo(self):
        try:
            p = float(self.txt_peso.get())
            e = float(self.txt_estatura.get())

            if p <= 0 or e <= 0:
                self.lbl_status.config(text="⚠  Error: Lo
                return

            imc = p / (e ** 2)
            res = f"=== CÁLCULO DE IMC (EVENTO PROCESADO)
            res += f"Peso: {p} kg | Estatura: {e} m\n"
            res += f"IMC: {imc:.2f}\n"

            self.txt_salida.delete("1.0", tk.END)
            self.txt_salida.insert(tk.END, res)
            self.lbl_status.config(text="✓ Cálculo realiz

        except ValueError:
            self.lbl_status.config(text="⚠  Error: Ingres

    def limpiar_campos(self):
        self.txt_peso.delete(0, tk.END)
        self.txt_estatura.delete(0, tk.END)
        self.txt_salida.delete("1.0", tk.END)
        self.lbl_status.config(text="Estado: Campos limpi

if __name__ == "__main__":
    app = VentanaEventos()
    app.mainloop()
```

2.3 Diagramas UML (Ejercicio 8.3)

Diagrama de Clases UML

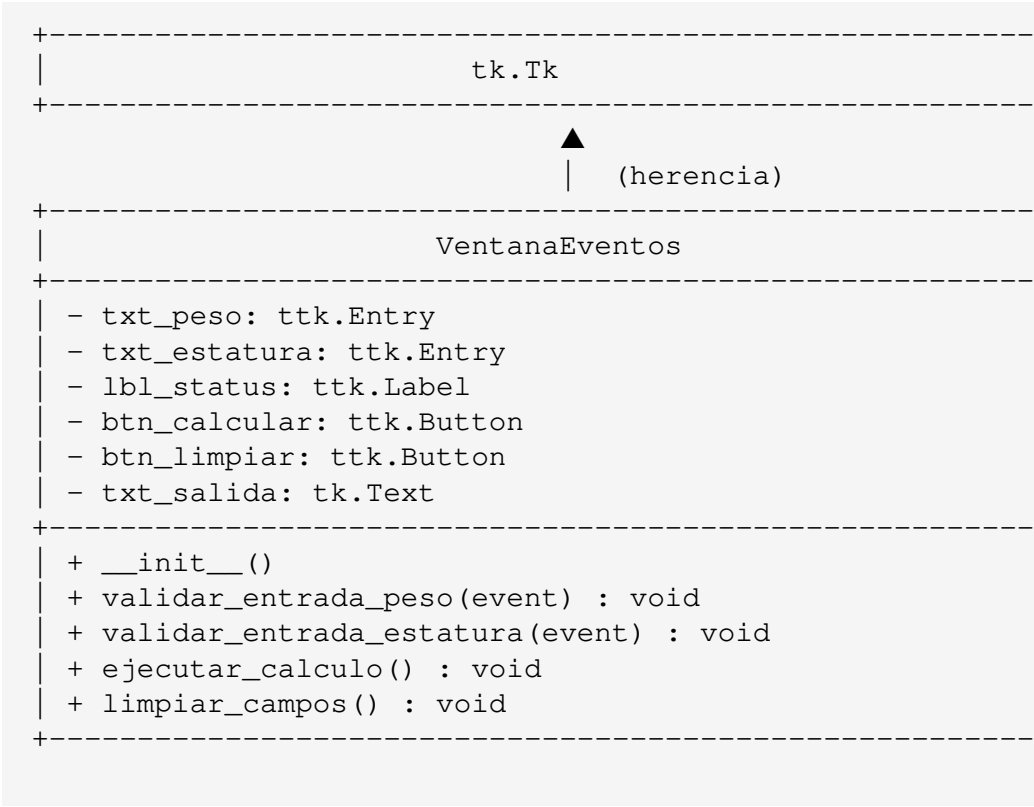
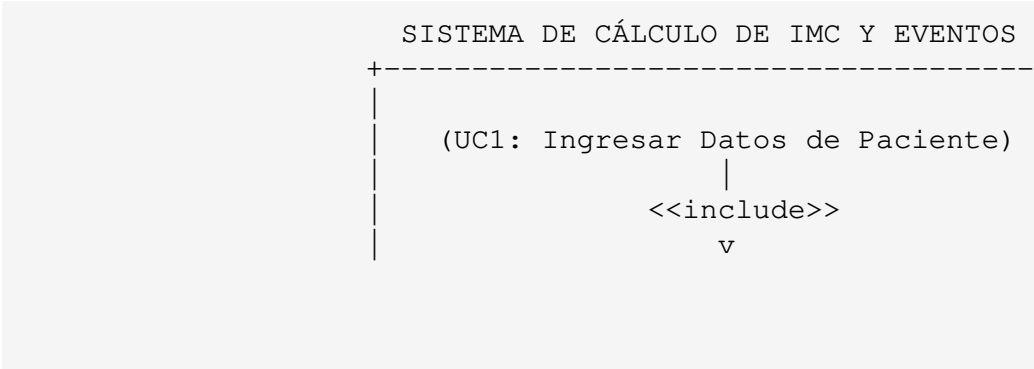
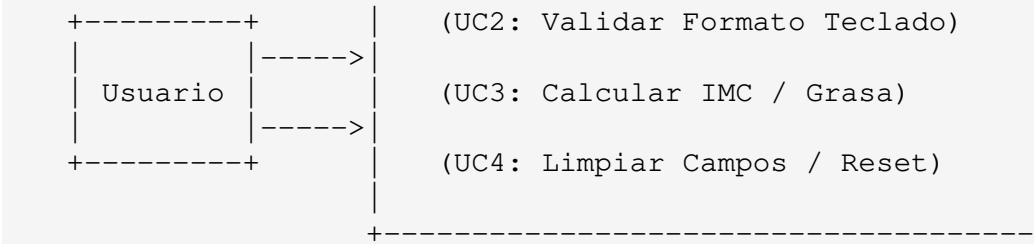


Diagrama de Casos de Uso UML





3. ENTREGABLES Y REPOSICIONAMIENTO EN GITHUB

- **Repositorio General del Curso**:

  https://github.com/Kevin007ch/POO-Kevin-Santiago-Montoya-Chavarri